



INSTITUTO POLITÉCNICO NACIONAL



Unidad Profesional Interdisciplinaria en Ingeniería y
Tecnologías Avanzadas

Tarea 3:

fragmentación horizontal

Alumnos:

Gamez Grees Isaac Humberto

Martínez Barrueta Mariana

Zuñiga Áviles Nathan Emiliano

Base De Datos Distribuidas

Fragmentación horizontal

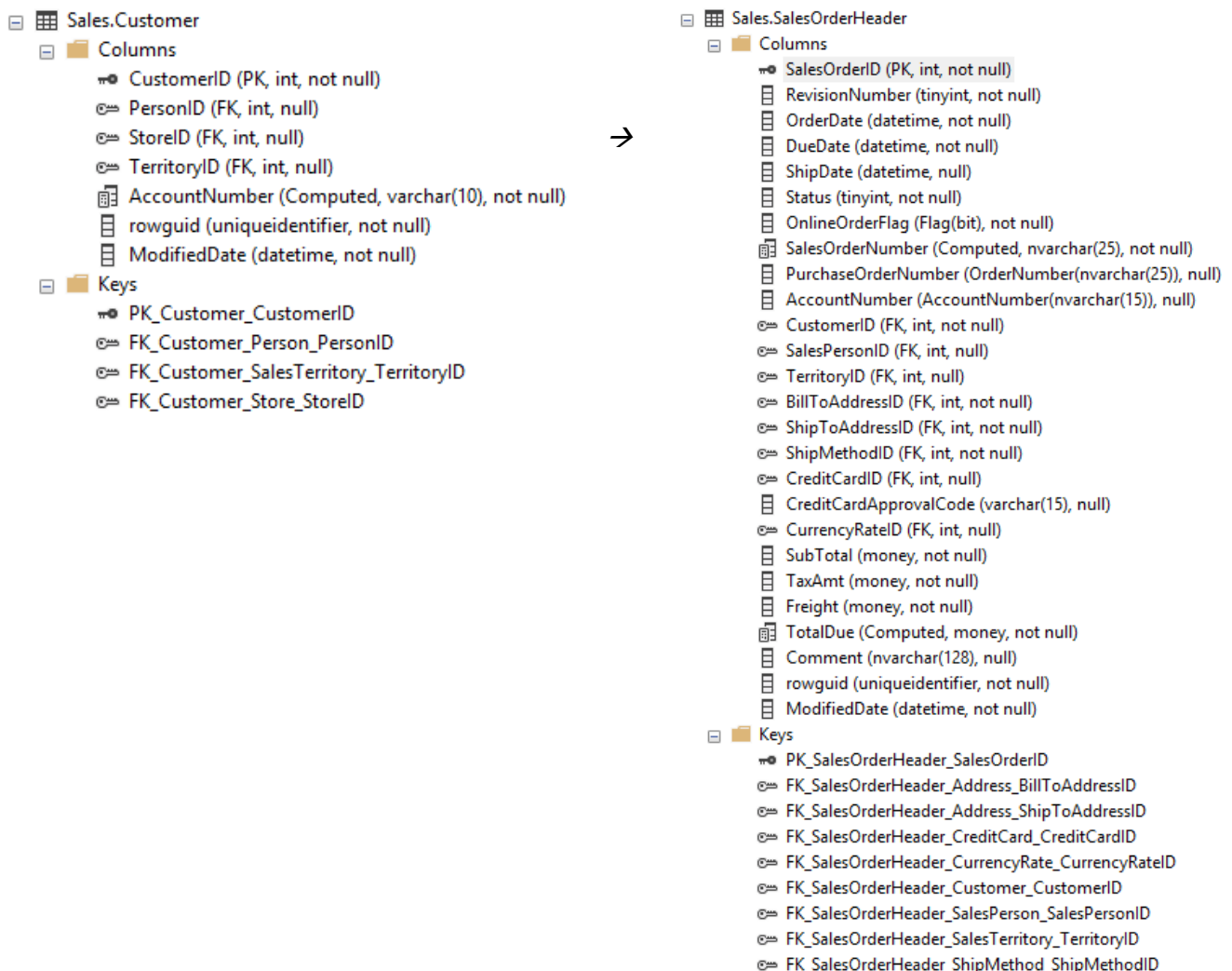
Ejercicio 3: Distribuir la cartera de clientes en los 5 nodos geográficos para que cada región administre y consulte de forma autónoma los datos de sus propios compradores. Tabla Sales.Customer, columna CustomerId (rangos).

En este ejercicio la tabla **Sales.Customer** será nuestra **tabla propietaria**, a partir de la cual se va a hacer la fragmentación horizontal primaria. Esto significa que se utilizarán atributos de la misma tabla para dividir registros; en este caso, el atributo que se tomará como referencia será **CustomerId**.

A partir de la tabla propietaria **Sales.Customer**, se identificaron las tablas relacionadas con clientes y que posteriormente pueden participar en la generación de fragmentos derivados.

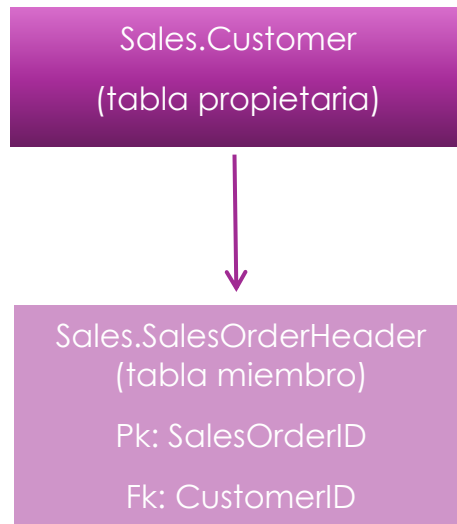
La primera relación se presenta con la tabla **Sales.SalesOrderHeader**, la cual contiene las órdenes realizadas por los clientes. La tabla de **Sales.Customer** tiene como llave **primaria CustomerID**, mientras que **Sales.SalesOrderHeader** tiene como llave **primaria SalesOrderID** y además contiene **CustomerId**, entonces se puede representar como:

Customer.CustomerID -> SalesOrderHeader.CustomerID



Esto quiere decir que un cliente puede tener una o varias órdenes de venta. Por esta razón, **Sales.SalesOrderHeader** funciona como una **tabla miembro** respecto a **Sales.Customer**.

Grafo relacional:



Para determinar los rangos que se utilizarán en la fragmentación horizontal primaria, primero se realizó un análisis del atributo **CustomerID** de la tabla **Sales.Customer**. Se obtuvo como valor mínimo 1, como valor máximo 30118 y un total de 19820 registros.

```
SELECT
    MIN(CustomerID) AS MinimoID,
    MAX(CustomerID) AS MaximoID,
    COUNT(*) AS TotalClientes
FROM Sales.Customer;
```

	MinimoID	MaximoID	TotalClientes
1	1	30118	19820

Primero sabemos que tenemos 19,820 clientes. Como necesitamos distribuirlos en cinco nodos, buscamos que cada fragmento tenga aproximadamente 3,964 clientes. Posteriormente se verificó la cantidad de registros correspondiente a cada rango de **CustomerID** utilizando **COUNT(*)**.

```
SELECT COUNT(*) AS ClientesGrupo1
FROM Sales.Customer
WHERE CustomerID BETWEEN 1 AND 14262;
SELECT COUNT(*) AS ClientesGrupo1
FROM Sales.Customer
WHERE CustomerID BETWEEN 1 AND 14262;
SELECT COUNT(*) AS ClientesGrupo3
FROM Sales.Customer
WHERE CustomerID BETWEEN 18227 AND 22190;
SELECT COUNT(*) AS ClientesGrupo4
FROM Sales.Customer
WHERE CustomerID BETWEEN 22191 AND 26154;
SELECT COUNT(*) AS ClientesGrupo5
FROM Sales.Customer
WHERE CustomerID BETWEEN 26155 AND 30118;
```

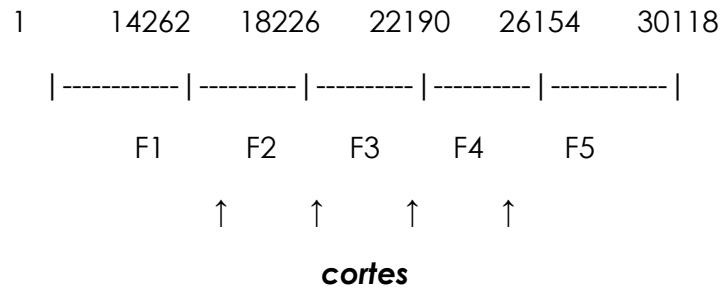
	ClientesGrupo1
1	3964

	ClientesGrupo1
1	3964

	ClientesGrupo3
1	3964

	ClientesGrupo4
1	3964

	ClientesGrupo5
1	3964



Posteriormente, se definieron los **predicados simples**, los cuales representan **las condiciones que se aplicaran sobre los datos**. A partir de los puntos de corte obtenidos del análisis anterior, se establecieron los siguientes predicados:

p1: Customer ID $\leq$ 14262

p2: Customer ID $\leq$ 18226

p3: Customer ID $\leq$ 22190

4: Customer ID $\leq$ 26154

Conjunto de predicados Pr : {p1,p2,p3,p4}

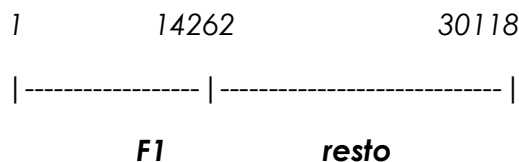
Una vez definido el conjunto inicial de predicados Pr , se aplicó el algoritmo **COM_MIN**, cuyo objetivo es obtener un conjunto de predicados que sea completo y mínimo.

Un conjunto es **completo** cuando los predicados deben ser suficientes para separar la tabla en los grupos que necesitamos. Y es **mínimo** cuando todos los predicados que contiene son relevantes para la fragmentación. Si eliminamos uno y ese predicado no cambia realmente la manera en que se separan los datos, entonces sobraba, es decir, no debemos tener predicados de más.

Iteración 1:

p1: Customer ID $\leq$ 14262

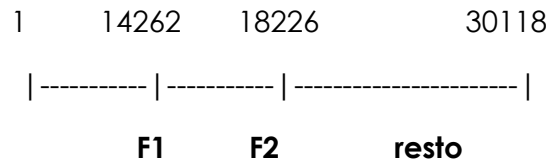
Ese predicado divide **Sales.Customer** en $CustomerID \leq 14262$ y $CustomerID > 14262$.



Entonces sí produce una separación necesaria, **p1 es relevante**. Por lo tanto, **$Pr'=\{P1\}$**

Iteración 2:

P2 nos permite dividir el bloque que había quedado después de 14262:



Ahora tenemos:

$F1: CustomerID \leq 14262$

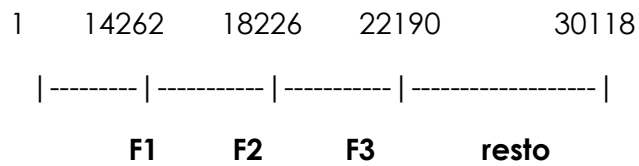
$F2: CustomerID > 14262 \wedge CustomerID \leq 18226$

Tenemos:

$Pr'=\{P1,P2\}$

Iteración 3:

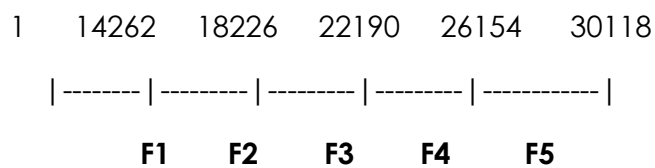
$P3: CustomerID \leq 22190$



$F3: CustomerID > 18226 \wedge CustomerID \leq 22190$

Iteración 4:

$P4: CustomerID \leq 26154$



$F4: CustomerID > 22190 \wedge CustomerID \leq 26154$

Y todo lo que quede después pertenece a:

$F5: CustomerID > 26154$

Finalmente, el resultado de COM_MIN:

$$Pr'=\{P1,P2,P3,P4\}$$

Donde:

$$P1: \text{Customer ID} \leq 14262$$

$$P2: \text{Customer ID} \leq 18226$$

$$P3: \text{Customer ID} \leq 22190$$

$$P4: \text{Customer ID} \leq 26154$$

porque si eliminamos cualquiera, perdemos uno de nuestros puntos de corte, además los cuatro predicados son relevantes y ninguno sobra.

Una vez obtenido mediante COM_MIN el conjunto completo y mínimo de predicados, se procedió a generar los fragmentos horizontales primarios de la relación Sales.Customer.

Para ello se combinaron los predicados y sus negaciones con la finalidad de delimitar cada uno de los cinco rangos establecidos previamente. Estas combinaciones permiten que cada registro de Sales.Customer pertenezca únicamente al fragmento que corresponda según el valor de su CustomerID.

Primer fragmento:

Necesitamos que el primer fragmento contenga todos los clientes hasta 14262.

$$F1=P1$$

$$F1: \text{CustomerID} \leq 14262$$

$$\text{Customer1} = \sigma \text{ CustomerID} \leq 14262 (\text{Sales.Customer})$$

Customer_1 contiene todos los registros de Sales.Customer cuyo CustomerID sea menor o igual a 14262.

Segundo fragmento:

Aquí queremos los clientes que **ya no pertenecen al primer fragmento**, pero que todavía cumplen P2.

Se utiliza $\neg P1 \wedge P2$ donde $\neg$ significa "NO". Entonces $\neg P1$ significa **CustomerID > 14262** y **P2** significa **CustomerID ≤ 18226**, al juntarlos **F2 = $\neg P1 \wedge P2$** y obtendremos:

$$14262 < \text{CustomerID} \leq 18226$$

$$\text{Customer2} = \sigma 14262 < \text{CustomerID} \leq 18226 (\text{Sales.Customer})$$

Tercer fragmento:

$$\text{Customer3} = \sigma 18226 < \text{CustomerID} \leq 22190 (\text{Sales.Customer})$$

Cuarto fragmento:

$Customer4 = \sigma_{22190 < CustomerID \leq 26154}(Sales.Customer)$

Quinto fragmento:

Aquí ya no tenemos otro punto de corte después de P4 pero necesitamos todo lo que **no cumple** P4:

$F5 = \neg P4 \rightarrow CustomerID > 26154$

$Customer5 = \sigma_{CustomerID > 26154}(Sales.Customer)$

Por lo tanto, los fragmentos primarios son:

$Customer1 = \sigma_{CustomerID \leq 14262}(Sales.Customer)$

$Customer2 = \sigma_{14262 < CustomerID \leq 18226}(Sales.Customer)$

$Customer4 = \sigma_{22190 < CustomerID \leq 26154}(Sales.Customer)$

$Customer4 = \sigma_{22190 < CustomerID \leq 26154}(Sales.Customer)$

$Customer5 = \sigma_{CustomerID > 26154}(Sales.Customer)$

Una vez obtenidos los cinco fragmentos primarios de **Sales.Customer**, se procedió a generar los fragmentos derivados de la tabla miembro **Sales.SalesOrderHeader**.

A diferencia de la fragmentación primaria, en este caso no se definieron nuevos rangos sobre el atributo **SalesOrderID**. Los fragmentos de **SalesOrderHeader** se obtuvieron a partir de los fragmentos previamente generados en **Sales.Customer**.

Para ello se utilizó la operación **SEMI-JOIN**, tomando como atributo de relación **CustomerID**. De esta manera, cada fragmento de **SalesOrderHeader** contiene únicamente las órdenes asociadas con los clientes pertenecientes al fragmento correspondiente de **Sales.Customer**, es decir, que las órdenes de cada cliente **sigan al fragmento donde quedó su cliente**.

El **SEMI-JOIN** nos ayuda a obtener los registros de una tabla miembro que corresponden a un fragmento de la tabla propietaria, es decir, que queremos seleccionar de una tabla solamente las filas que tengan relación con las filas de otra por medio de un atributo. El símbolo que lo representa es: $\bowtie$

La condición que relaciona las tablas es:

$SalesOrderHeader.CustomerID = Customer.CustomerID$

Por lo tanto:

$SOH2=SalesOrderHeader \bowtie Customer1$

$SOH2=SalesOrderHeader \bowtie Customer1$

$SOH3=SalesOrderHeader \bowtie Customer1$

$SOH4=SalesOrderHeader \bowtie Customer1$

$SOH5=SalesOrderHeader \bowtie Customer1$

Una vez generados los fragmentos primarios de **Sales.Customer** y los fragmentos derivados de **Sales.SalesOrderHeader**, se verificó que el diseño cumpliera con las tres reglas de fragmentación: completitud, reconstrucción y disyunción.

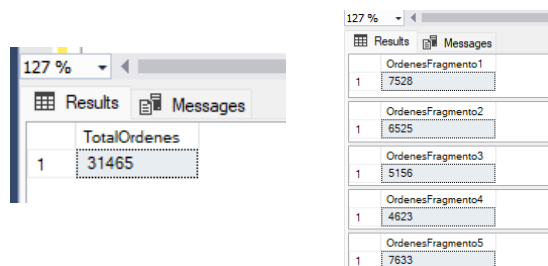
Completitud:

Esta regla establece que todos los registros de la relación original deben encontrarse incluidos en alguno de los fragmentos. En **Sales.Customer** se tienen 19,820 registros y cada uno de los cinco fragmentos contiene 3,964 clientes, por lo que:

$$3964+3964+3964+3964+3964=19820$$

De esta manera, ningún cliente queda fuera del diseño. Para Sales.SalesOrderHeader, la suma de los registros de los cinco fragmentos derivados es:

$$7528+6525+5156+4623+7633=31465$$



Results	
TotalOrdenes	31465

Results	
OrdenesFragmento1	7528
OrdenesFragmento2	6525
OrdenesFragmento3	5156
OrdenesFragmento4	4623
OrdenesFragmento5	7633

Reconstrucción:

Esta regla establece que debe ser posible reconstruir la relación original a partir de sus fragmentos mediante una **unión**.

Para **Sales.Customer**:

$Sales.Customer=Customer1 \cup Customer2 \cup Customer3 \cup Customer4 \cup Customer5$

Si volvemos a juntar los cinco fragmentos, debemos obtener nuevamente los 19,820 clientes originales. Entonces **$3964(5)=19820$** obtenemos nuevamente la tabla original.

Para **SalesOrderHeader**:

$$\text{SalesOrderHeader} = \text{SOH}_1 \cup \text{SOH}_2 \cup \text{SOH}_3 \cup \text{SOH}_4 \cup \text{SOH}_5$$

$$\text{SOH}_1 = 7528 + \text{SOH}_2 = 6525 + \text{SOH}_3 = 5156 + \text{SOH}_4 = 4623 + \text{SOH}_5 = 7633 = 31465$$

Y nuevamente obtenemos la tabla original.

Disyunción:

Esta regla establece que un registro no debe pertenecer simultáneamente a dos fragmentos horizontales diferentes. Es decir, queremos evitar duplicados entre fragmentos.

Por ejemplo, para **SalesCustomer** tenemos CustomerID = 15000, el cual esta únicamente en el fragmento F2 → > 14262 y <= 18226 y pertenece a Customer2.

Además, para **SalesOrderHeader** se conserva exactamente igual porque cada orden tiene un **CustomerID** y ese cliente solamente pertenece a **un fragmento** de **Customer**. Por ejemplo, tenemos CustomerID = 15000 → Customer2 → todas sus órdenes → SalesorderHeader2.

Con base en estas comprobaciones, el diseño propuesto cumple con las tres reglas requeridas para una fragmentación horizontal correcta.

Conclusiones:

Cada uno de estos fragmentos puede ser asignado a una región diferente para que cada nodo administre la información que le corresponde. Aunque los rangos se hicieron para que al inicio hubiera la misma cantidad de clientes en cada grupo, con el tiempo esto podría cambiar si se agregan nuevos registros, por lo que sería necesario revisar nuevamente los rangos.

En general, el diseño permitió organizar los datos de una forma más distribuida y entender cómo una tabla principal puede dividirse y cómo las tablas relacionadas pueden seguir esa misma división.